

MODULE 1

Cadrer avant de coder

Module 1. Cadrer avant de coder

Ce module pose les fondations du projet : initialisation sécurisée du dépôt, inventaire des actifs, modélisation des menaces STRIDE et matrice de risques. Tout est documenté dans `docs/` avant d'écrire une ligne de code correctif.

Objectifs opérationnels

À la fin de ce module, tu auras dans ton application :

- ☒ Un dépôt Git initialisé avec `.gitignore` et `.env.example` corrects
 - ☒ `docs/00-project-brief.md` décrivant l'app et son périmètre
 - ☒ `docs/01-risk-analysis.md` avec 6 actifs inventoriés
 - ☒ `docs/02-threat-model.md` avec le modèle STRIDE sur 3 composants
 - ☒ Une matrice de risques Impact × Probabilité documentée
 - ☒ Un commit tagué `M1-cadrage`
-

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
0	Intro + brief	Créer <code>docs/00-project-brief.md</code>	10 min
1	Security by Design	Init Git sécurisé (<code>.gitignore</code> , <code>.env.example</code>)	10 min
2	Inventaire des actifs	Lister 6 actifs dans <code>docs/01-risk-analysis.md</code>	15 min
3	Surface d'attaque	Lister endpoints publics / auth / admin	10 min
4	Threat model STRIDE	Appliquer sur 3 composants dans <code>docs/02-threat-model.md</code>	20 min
5	Matrice de risques + risques résiduels	Compléter la matrice, documenter les acceptés	15 min
6	Commit + tag	<code>git tag M1-cadrage</code>	10 min

Parcours

Parcours A, App personnelle : tu appliques toutes les étapes à ton propre projet (API, webapp, etc.).

Parcours B, App vulnérable fournie : tu travailles sur `vuln-app/` (TaskFlow, port 3000). C'est le parcours par défaut si tu n'as pas de projet personnel.

⚠ Les exemples de ce module utilisent TaskFlow comme fil conducteur.

CONCEPT 1, Security by Design

Rappel. “Sécuriser après coup” coûte 10× plus cher que d'intégrer la sécurité dès le départ. Security by Design signifie prendre des décisions sécurisées avant d'écrire du code : choix de bibliothèques, gestion des secrets, contrôle d'accès par défaut restrictif. La règle fondamentale : **par défaut, tout est interdit ; on ouvre ce qui est**

nécessaire. Un `.gitignore` oublié avant le premier commit peut exposer des secrets pour toujours dans l'historique Git.

Risque concret	Un fichier <code>.env</code> commité contenant des clés API ou un mot de passe de base de données est indexé par GitHub. Des scanners automatiques (TruffleHog, GitLeaks) le trouvent en moins d'une heure.
Exemple de faille	Secret commité par inadvertance (situation réelle)
À vérifier dans l'app	Est-ce que <code>.env</code> est dans <code>.gitignore</code> ? Est-ce que <code>db/*.db</code> est ignoré ? Est-ce qu'un <code>.env.example</code> existe ?
Correction	Créer <code>.gitignore</code> et <code>.env.example</code> avant tout commit
Test de validation	<code>git status</code> ne doit PAS lister <code>.env</code> ni <code>db/taskflow.db</code>
Trace rapport	“Le dépôt a été initialisé avec un <code>.gitignore</code> couvrant les fichiers <code>.env</code> , les bases SQLite et <code>node_modules</code> . Un <code>.env.example</code> documente les variables requises sans exposer de valeurs réelles.”

Exemple de fichier `.env` qui ne doit JAMAIS être commité :

```
# .env , NE PAS COMMITER
JWT_SECRET=secret123
DB_PASSWORD=Sa$uper123!
PORT=3000
```

Exemple de faille (historique Git) :

```
# Quelqu'un a commité .env par erreur, puis l'a supprimé
git log --all --full-history -- .env
# Le fichier est toujours accessible via :
git show abc1234:.env
# => JWT_SECRET=secret123 est visible pour toujours
```

TP 1, Init Git sécurisé (10 min)

ÉTAPE 1.1, CRÉER .GITIGNORE

```
# À la racine du projet (ou dans vuln-app/)
cat > .gitignore << 'EOF'
# Secrets et configuration locale
.env
.env.local
.env.*.local

# Base de données SQLite
db/*.db
db/*.db-wal
db/*.db-shm

# Dépendances
node_modules/

# Fichiers uploadés
uploads/

# Logs
*.log
logs/

# OS
.DS_Store
Thumbs.db
EOF
```

ÉTAPE 1.2, CRÉER .ENV.EXAMPLE

```
cat > .env.example << 'EOF'
# Copier ce fichier en .env et remplir les valeurs
PORT=3000
JWT_SECRET=remplacer_par_une_valeur_forte_32_chars_min
NODE_ENV=development
EOF
```

ÉTAPE 1.3, VÉRIFICATION

```
git status
# .env ne doit PAS apparaître dans la liste
# .env.example DOIT apparaître (nouveau fichier non-ignoré)

# Si .env est déjà traqué par erreur :
git rm --cached .env
git commit -m "fix: remove .env from tracking"
```

✓ **Point de vérification :** `git status` ne liste ni `.env` ni `db/taskflow.db`.

CONCEPT 2, Inventaire des actifs

Rappel. On ne peut pas protéger ce qu'on ne connaît pas. Un actif est tout élément qui a de la valeur et qui pourrait être compromis : données, services, secrets, code source. L'inventaire est la première étape de tout audit. Pour chaque actif, on identifie sa valeur (pourquoi il est critique), sa localisation dans le système et son propriétaire fonctionnel. Un actif non inventorié est un actif non protégé.

Risque concret	Sans inventaire, on oublie de protéger la table <code>users</code> ou les fichiers uploadés. L'attaquant cible précisément ce qu'on a oublié de sécuriser.
Exemple de faille	Fichier <code>db/taskflow.db</code> accessible depuis le navigateur car dans un dossier servi en statique
À vérifier dans l'app	Quelles tables SQLite ? Quels fichiers ? Quels secrets ? Quels services tiers ?
Correction	Documenter dans <code>docs/01-risk-analysis.md</code>
Test de validation	Parcourir la liste et confirmer qu'aucun actif critique n'est exposé publiquement
Trace rapport	"L'inventaire des actifs a identifié 6 éléments critiques. La base SQLite <code>taskflow.db</code> et les variables d'environnement <code>JWT_SECRET</code> constituent les actifs à plus haute valeur. Aucun n'est exposé directement au réseau dans la version corrigée."

TP 2, Inventaire des actifs (15 min)

ÉTAPE 2.1, CRÉER `DOCS/01-RISK-ANALYSIS.MD`

```
mkdir -p docs
```

Contenu à rédiger (adapte les valeurs à ton app) :

```
# Analyse de risques, TaskFlow

## 1. Inventaire des actifs

| # | Actif | Type | Localisation | Valeur | Propriétaire |
|---|---|---|---|---|---|
| A1 | Base de données `taskflow.db` | Donnée | `db/taskflow.db` | Critique | Dev/Ops |
| A2 | Mots de passe utilisateurs | Donnée | Table `users.password` | Critique | Métier |
| A3 | Tokens JWT | Secret | Mémoire + cookies client | Élevée | Dev |
| A4 | Variable `JWT_SECRET` | Secret | `.env` / code source | Critique | Dev |
| A5 | Fichiers uploadés | Donnée | `uploads/` | Moyenne | Utilisateur |
| A6 | Code source de l'API | Logiciel | `server/` | Élevée | Dev |
```

ÉTAPE 2.2, SURFACE D'ATTAQUE : LISTER LES ENDPOINTS

Ouvre `server/index.js` et les fichiers dans `server/routes/` :

```
# Lister tous les endpoints déclarés
grep -r "router\.\(get\|post\|put\|delete\|patch\)" vuln-app/server/routes/
```

Résultat attendu pour TaskFlow :

```
## 2. Surface d'attaque

| Endpoint | Méthode | Accès | Données traitées |
|---|---|---|---|
| /auth/register | POST | Public | email, password |
| /auth/login | POST | Public | email, password |
| /tasks | GET | Auth | JWT token |
| /tasks | POST | Auth | title, content |
| /tasks/:id | PUT | Auth | title, content |
| /tasks/:id | DELETE | Auth | id tâche |
| /admin/users | GET | Admin | tous les users |
| /upload | POST | Auth | fichier |
```

✓ **Point de vérification** : chaque endpoint est classé Public / Auth / Admin.

CONCEPT 3, Surface d'attaque

Rappel. La surface d'attaque est l'ensemble des points d'entrée qu'un attaquant peut utiliser pour interagir avec le système. Plus la surface est grande, plus le risque est élevé. On réduit la surface en désactivant ce qui n'est pas nécessaire, en restreignant l'accès aux endpoints sensibles et en validant strictement toutes les entrées. Un endpoint `/admin` accessible sans authentification multiplie la surface par 10.

Risque concret	<code>GET /admin/users</code> retourne tous les comptes si le middleware d'auth est absent ou bypassable.
Exemple de faille	Endpoint admin sans vérification de rôle (voir <code>server/routes/admin.js</code>)
À vérifier dans l'app	Chaque route admin vérifie-t-elle le rôle ? Les routes publiques valident-elles les entrées ?
Correction	Classifier et documenter, la correction effective est en M3 (JWT)
Test de validation	<code>curl http://localhost:3000/admin/users</code> sans token doit retourner 401
Trace rapport	“L'analyse de la surface d'attaque a identifié 8 endpoints. Parmi eux, <code>/admin/users</code> était accessible sans contrôle de rôle dans la version initiale. La surface a été réduite par l'ajout de middlewares d'authentification (corrigé en M3).”

CONCEPT 4, Threat Model STRIDE

Rappel. STRIDE est un cadre mnémotechnique pour identifier les menaces : **S**poofing (usurpation d'identité), **T**ampering (altération de données), **R**epudiation (nier une action), **I**nformation Disclosure (fuite d'information), **D**enial of Service (dénier de service), **E**levation of Privilege (élévation de privilèges). Pour chaque composant critique, on se demande : “Comment un attaquant pourrait-il exploiter cette catégorie ?” Le résultat est un tableau de menaces prioritaires.

LETTRE	MENACE	EXEMPLE SUR TASKFLOW
S	Spoofing	Usurper le compte admin si le JWT est forgeable
T	Tampering	Modifier la tâche d'un autre utilisateur via PUT /tasks/:id
R	Repudiation	Aucun log des actions → impossible de prouver qui a supprimé une tâche
I	Information Disclosure	/admin/users expose emails + mots de passe en clair
D	DoS	Brute force illimité sur /auth/login sans rate limit
E	Elevation of Privilege	Changer son rôle de user à admin en forgeant le payload JWT
Risque concret		Sans threat model, les corrections sont réactives. On corrige ce qu'on a trouvé, pas ce qui est le plus dangereux.
Exemple de faille		JWT signé avec secret123 → un attaquant peut forger un token role: "admin"
À vérifier dans l'app		Appliquer STRIDE sur : (1) authentification, (2) gestion des tâches, (3) interface admin
Correction		Documenter dans docs/02-threat-model.md , les corrections viendront dans M2, M3, M4
Test de validation		Le document docs/02-threat-model.md contient au moins 3 composants × 6 catégories STRIDE
Trace rapport		“Le threat model STRIDE a été appliqué sur 3 composants critiques (auth, tasks, admin). 14 menaces identifiées, dont 4 de criticité Haute. Les menaces E (Elevation of Privilege via JWT forgeable) et D (DoS par brute force) ont été priorisées en Po.”

TP 3, STRIDE sur 3 composants (20 min)

ÉTAPE 3.1, CRÉER DOCS/02-THREAT-MODEL.MD

```
# Threat Model STRIDE, TaskFlow

## Composant 1 : Authentification (`/auth/register`, `/auth/login`)

| STRIDE | Menace identifiée | Probabilité | Impact | Priorité |
|-----|-----|-----|-----|-----|
| S | Usurpation via token JWT forgé (secret faible) | Haute | Critique | P0 |
| T | Altération du payload JWT (rôle user→admin) | Haute | Critique | P0 |
| R | Aucun log des connexions réussies/échouées | Moyenne | Moyenne | P2 |
| I | Fuite de mots de passe en clair dans les logs | Haute | Critique | P0 |
| D | Brute force illimité sur /login | Haute | Haute | P1 |
| E | Escalade de privilèges via JWT manipulé | Haute | Critique | P0 |

## Composant 2 : Gestion des tâches (`/tasks`)

| STRIDE | Menace identifiée | Probabilité | Impact | Priorité |
|-----|-----|-----|-----|-----|
| S | Accès aux tâches d'un autre utilisateur (IDOR) | Haute | Haute | P0 |
| T | Modification de tâche appartenant à autrui | Haute | Haute | P0 |
| R | Aucun audit trail sur la suppression de tâches | Moyenne | Faible | P3 |
| I | Tâches privées visibles via /admin/users | Haute | Haute | P1 |
| D | Création de tâches en masse (pas de limite) | Faible | Moyenne | P3 |
| E | Lecture tâches admin via manipulation user_id | Moyenne | Haute | P1 |

## Composant 3 : Interface Admin (`/admin/users`)

| STRIDE | Menace identifiée | Probabilité | Impact | Priorité |
|-----|-----|-----|-----|-----|
| S | Accès sans authentification si middleware absent | Haute | Critique | P0 |
| T | Modification de rôle directement en BDD | Faible | Critique | P1 |
| R | Aucun log des actions admin | Moyenne | Haute | P2 |
| I | Exposition de tous les emails + mots de passe | Haute | Critique | P0 |
| D | Suppression en masse d'utilisateurs | Faible | Critique | P2 |
| E | Admin peut lire tokens JWT actifs d'autres users | Faible | Haute | P2 |
```

✓ **Point de vérification** : le document contient 3 tableaux remplis, au moins 2 menaces Po.

CONCEPT 5, Matrice de risques

Rappel. La matrice de risques combine deux axes : **Impact** (conséquence si la menace se réalise) et **Probabilité** (chance que la menace se réalise). Le produit des deux donne une criticité qui permet de prioriser les corrections. On utilise généralement une échelle

1-3 ou 1-5. Ce qui est critique (Impact Haute × Probabilité Haute) se corrige en premier. Ce qui est faible peut être accepté ou mitigé plus tard.

Risque concret	Sans priorisation, on passe du temps sur des risques faibles pendant que des risques critiques restent ouverts.
Exemple de faille	Corriger les headers HTTP (impact moyen) avant de hasher les mots de passe (impact critique) serait une erreur de priorisation.
À vérifier dans l'app	Toutes les menaces STRIDE sont-elles dans la matrice ? Les Po sont-elles bien identifiées ?
Correction	Compléter la section Matrice dans <code>docs/01-risk-analysis.md</code>
Test de validation	Chaque menace Po du threat model a une ligne dans la matrice avec un responsable et une échéance
Trace rapport	“La matrice de risques identifie 4 risques Critiques (mot de passe en clair, JWT forgeable, brute force login, IDOR sur /tasks). Ces 4 risques ont été traités en priorité dans les modules M2, M3 et M4.”

ÉTAPE 4.1, AJOUTER LA MATRICE DANS `DOCS/01-RISK-ANALYSIS.MD`

3. Matrice de risques

#	Menace	Impact (1-3)	Probabilité (1-3)	Score	Priorité	Module
R1	Mots de passe en clair	3	3	9	P0	M2
R2	JWT secret faible + forgeable	3	3	9	P0	M3
R3	Brute force /login sans rate limit	3	3	9	P0	M2
R4	IDOR sur /tasks (accès tâches d'autrui)	3	2	6	P1	M4
R5	Admin sans contrôle de rôle	3	2	6	P1	M3
R6	Injection SQL sur /tasks	3	2	6	P1	M4
R7	Upload sans validation de type	2	2	4	P2	M5
R8	Headers HTTP manquants (helmet)	1	3	3	P2	M5
R9	Stack trace exposée au client	2	3	6	P1	M6

CONCEPT 6, Risques résiduels

Rappel. On ne corrige jamais 100% des risques. Certains sont acceptés parce que le coût de correction dépasse l'impact probable (faible risque, correction complexe), ou parce que la contrainte métier l'impose (ex: pas de 2FA car les utilisateurs ne le veulent pas). Documenter les risques résiduels est une obligation professionnelle : cela prouve que la décision d'accepter un risque a été consciente, pas ignorée. Un risque résiduel non documenté est une faute professionnelle.

Risque concret	Dans un audit, ne pas documenter les risques acceptés crée une ambiguïté : est-ce oublié ou décidé ?
Exemple de faille	Absence de 2FA documentée comme "hors périmètre" vaut mieux que silence total
À vérifier dans l'app	Y a-t-il des menaces STRIDE pour lesquelles on n'a pas de correction dans ce cours ?
Correction	Ajouter une section "Risques résiduels" dans <code>docs/01-risk-analysis.md</code>
Test de validation	Chaque risque résiduel a une justification et un responsable de la décision d'acceptation
Trace rapport	"Les risques résiduels suivants ont été explicitement acceptés pour ce sprint : absence de 2FA (hors périmètre M1), pas d'audit trail complet (reporté). Ces décisions sont documentées et seront réévaluées à chaque sprint."

ÉTAPE 5.1, AJOUTER LA SECTION DANS `DOCS/01-RISK-ANALYSIS.MD`

```
## 4. Risques résiduels
```

```
| # | Risque | Raison d'acceptation | Propriétaire | Réévaluation |
|---|---|---|---|---|
| RR1 | Absence de 2FA | Hors périmètre du cours | Toi | Fin M7 |
| RR2 | Pas d'audit trail complet | Complexité SQLite > bénéfice | Dev | Sprint 2 |
| RR3 | Pas de WAF | Infrastructure hors scope | Ops | Production |
```

TP final, Commit + tag (10 min)

```
# Depuis la racine du projet
git add docs/ .gitignore .env.example
git status # Vérifier ce qui sera commité

git commit -m "M1: cadrage sécurisé, inventaire actifs + STRIDE + matrice risques"

git tag M1-cadrage
git log --oneline -3 # Vérifier que le tag est bien posé
```

✓ Point de vérification :

```
git tag
# Doit afficher : M1-cadrage
git show M1-cadrage --stat
# Doit lister : docs/00-project-brief.md, docs/01-risk-analysis.md, docs/02-threat-
model.md
```

Mini-quiz du module (8 QCM)

Q1. Que signifie le “D” dans STRIDE ?

- A. Data Exposure
- B. Denial of Service
- C. Directory Traversal
- D. Data Tampering

Q2. Pourquoi un `.gitignore` doit-il être créé avant le premier commit ?

- A. Pour améliorer les performances Git
- B. Pour que Git ignore les fichiers de cache
- C. Parce qu’une suppression ultérieure ne retire pas le fichier de l’historique
- D. Pour éviter les conflits de merge

Q3. Dans une matrice de risques, un risque avec Impact=3 et Probabilité=1 vaut :

- A. 1
- B. 3
- C. 4
- D. 9

Q4. Security by Design signifie principalement :

- A. Avoir un firewall devant l'application
- B. Intégrer la sécurité dès la conception plutôt qu'après développement
- C. Utiliser HTTPS
- D. Faire un audit de sécurité annuel

Q5. Un risque résiduel documenté est :

- A. Un risque ignoré
- B. Un risque qu'on n'a pas su corriger
- C. Un risque consciemment accepté avec justification
- D. Un risque de faible impact uniquement

Q6. Dans STRIDE, "E" (Elevation of Privilege) sur TaskFlow correspond à :

- A. Un utilisateur qui crée trop de tâches
- B. Un attaquant qui forge un JWT avec `role: "admin"`
- C. Un administrateur qui lit les logs
- D. Une fuite de la base de données

Q7. L'inventaire des actifs sert à :

- A. Estimer le coût du projet
- B. Identifier ce qui doit être protégé et pourquoi
- C. Lister les développeurs du projet
- D. Documenter les fonctionnalités

Q8. La surface d'attaque se réduit en :

- A. Ajoutant des fonctionnalités sécurisées
- B. Désactivant les endpoints inutiles et restreignant les accès
- C. Utilisant uniquement HTTPS
- D. Ajoutant de la journalisation

Corrigé

Q	RÉPONSE	POURQUOI
Q1	B	DoS = Denial of Service, rendre le service indisponible
Q2	C	L'historique Git conserve tous les fichiers jamais commités, même supprimés ensuite
Q3	B	Impact $\times$ Probabilité = $3 \times 1 = 3$
Q4	B	Security by Design = sécurité intégrée dès la conception
Q5	C	Risque accepté consciemment $\neq$ risque ignoré
Q6	B	Forger un JWT avec rôle élevé = Elevation of Privilege
Q7	B	Sans inventaire, on ne sait pas quoi protéger
Q8	B	Moins de points d'entrée = surface réduite

Livrable du module

- ✓ `.gitignore` couvrant `.env`, `*.db`, `node_modules/`, `uploads/`
- ✓ `.env.example` avec toutes les variables documentées
- ✓ `docs/00-project-brief.md` (nom app, routes principales, stack technique)
- ✓ `docs/01-risk-analysis.md` (6 actifs + matrice de risques + risques résiduels)
- ✓ `docs/02-threat-model.md` (STRIDE sur 3 composants, 18+ menaces)
- ✓ Commit avec tag `M1-cadrage`

Vérification finale :

```
git tag                # affiche M1-cadrage
ls docs/               # affiche 00-project-brief.md, 01-risk-
analysis.md, 02-threat-model.md
git show --stat M1-cadrage # liste les fichiers du commit tagué
```

Erreurs fréquentes à éviter

-  **Committer `.env` avant `.gitignore`** : l'historique conserve le secret pour toujours.
Correction : `git rm --cached .env` + `rewrite history` (complexe, éviter dès le départ).
-  **STRIDE rempli de façon mécanique** : cocher toutes les cases sans réfléchir au contexte de l'app. Chaque ligne doit être spécifique à TaskFlow, pas générique.
-  **Matrice sans priorités** : identifier les risques sans les ordonner est inutile. Toujours finir par “qu'est-ce qu'on corrige d'abord ?”.
-  **Risques résiduels vides** : une liste vide signifie soit qu'on a tout corrigé (rare), soit qu'on a oublié de documenter les acceptations. Les deux interprétations sont problématiques.
-  **Confondre actif et fonctionnalité** : un actif est quelque chose qui a de la valeur (données, secrets, code), pas une fonctionnalité (“le formulaire de login”).

Ressources

- [OWASP Threat Modeling](#)
- [OWASP STRIDE](#)
- [OWASP Top 10](#)
- [Microsoft Threat Modeling Tool](#)
- [gitignore.io](#), générateur de `.gitignore`
- [TruffleHog](#), scanner de secrets dans Git